

Sistemas Operacionais

Processo e Threads

04

Threads: Conceito e Vantagens

Conceitos, modelos e relação dos Threads

Threads

- Uma thread é a unidade básica de execução de um programa.
- Ela é um processo leve que pode ser considerado como um "subprocesso" dentro de um processo maior.
- As threads permitem que um programa execute várias tarefas de forma concorrente tornando-o mais eficiente e responsivo.

Threads

Os principais aspectos relacionados ao conceito de threads:

- **Execução concorrente:** as threads permitem que partes diferentes de um programa sejam executadas concorrentemente, o que pode melhorar o desempenho e a capacidade de resposta do programa.
- **Compartilhamento de recursos:** as threads em um mesmo processo compartilham o mesmo espaço de memória o que facilita o compartilhamento de dados e recursos entre elas.

Threads

- **Criação de threads:** os sistemas operacionais e as linguagens de programação geralmente fornecem mecanismos para criar e gerenciar threads. As threads podem ser criadas em um processo para realizar tarefas específicas.
- **Threads e programação paralela:** A programação com threads é uma forma de programação paralela que envolve a execução de várias tarefas simultaneamente para aproveitar melhor o poder de processamento de sistemas com múltiplos núcleos de CPU.

Threads

- **Dois tipos de threads:** threads de usuário e threads do kernel.
- Threads de usuário são criadas e gerenciadas pela biblioteca de threads da linguagem de programação ou pela aplicação em si.
- Threads do kernel são threads que são gerenciadas pelo próprio kernel do sistema operacional.
- Concorrência e sincronização: Ao usar threads é importante lidar com questões de concorrência como a sincronização de acesso a recursos compartilhados.
- Escalonamento de threads: O sistema operacional é responsável pelo escalonamento das threads, decidindo quais threads devem ser executadas e em que ordem.

Modelos de Threads

- Em sistemas operacionais os modelos de threads referem-se as diferentes abordagens usadas para criar, gerenciar e coordenar threads (ou processos leves) dentro de um processo maior.
- Cada modelo tem suas próprias características e trade-offs e a escolha do modelo apropriado depende dos requisitos específicos do sistema e do aplicativo.

Modelos de Threads

- Many-to-One (Muitos-para-Um): várias threads de nível de usuário são mapeadas para uma única thread de nível de kernel.
- A implementação das threads de nível de usuário é feita inteiramente no espaço do processo do usuário e o sistema operacional não tem conhecimento das threads de nível de usuário.
- One-to-One (Um-para-Um): cada thread de nível de usuário é mapeada para uma única thread de nível de kernel.
- Cada thread de nível de usuário é tratada como um processo separado pelo sistema operacional.

Modelos de Threads

- Many-to-Many (Muitos-para-Muitos): várias threads de nível de usuário são mapeadas para um número menor de threads de nível de kernel.
- A correspondência entre threads de nível de usuário e threads de nível de kernel é flexível.
- Two-Level (Dois Níveis): combina aspectos dos modelos many-to-one e one-to-one.
- As threads de nível de usuário são mapeadas para threads de nível de kernel mas de forma mais flexível do que a correspondência um-para-um.

Relação entre Threads e Processos

- Um thread é uma unidade básica de processamento que pode ser executada simultaneamente com outros threads dentro do mesmo processo.
- Threads compartilham o mesmo espaço de endereço de memória e recursos do sistema, o que os torna mais eficientes em termos de uso de recursos do que processos independentes.
- Um processo é um programa em execução.
- Ele contém o código executável, dados e recursos do sistema, como arquivos abertos e conexões de rede.

Relação entre Threads e Processos

- Os sistemas operacionais modernos usam uma combinação de processos e threads para gerenciar tarefas e recursos.
- O escalonamento de processos e threads é feito pelo sistema operacional para garantir que cada tarefa receba a quantidade necessária de recursos e seja executada de forma eficiente.